Week 3 Tutorials

Objective

The objective of this tutorial is to explore the query optimization techniques used in different kinds of SQL statements

- **AND** and **OR** logical operators
- Order of **JOIN**
- Nested queries (subqueries)
- Sargable queries

We will use 2 versions of the same 'company' database to experiment.

- Company database with keys
- Company database without keys

Exercise 1: Compare **AND** and **OR**

Use the **company** and **company_no_key** databases to analyze the following queries:

- Get all female employees who are in department #5
- Get all female employees and employees in department #5

Use the **EXPLAIN** (or **EXPLAIN ANALYZE**) statement to see execution plans. Analyze the similarities and differences between those execution plans in the **company** and **company_no_key** databases.

How do you improve the performance of those queries?

Change the order of conditions and observe the new result

```
SELECT *
FROM Employee
WHERE <condition 1> AND <condition 2>;

SELECT *
FROM Employee
WHERE <condition 2> AND <condition 1>;
```

Exercise 2: Order of JOIN

When you join 3 tables: **a**, **b**, **c** (assume there are common columns between **a** and **b**, there are common columns between **b** and **c**), there are 2 possible join orders: **(a JOIN b) JOIN c**, and **(b JOIN c) JOIN a**. The join orders can affect the performance of your queries. For example, **a**, **b**, **c** onto **COMPANY**: **a = DEPARTMENT**, **b = EMPLOYEE**, **c = DEPENDENT**. **DEPARTMENT** meets **EMPLOYEE** on **Dnumber = Dno**; **EMPLOYEE** meets **DEPENDENT** on **Ssn = Essn**.

- Write a query to display all employee names, department names, and dependent names for every employee who has at least one dependent
- Write the above query in 2 different join orders

Use the **EXPLAIN** (or **EXPLAIN ANALYZE**) statement to see execution plans. Analyze the similarities and differences between those plans in the **company** and **company_no_key** databases.

Exercise 3: Nested-queries/Sub-queries

Nested queries are those whose results are used by other queries. Usually, nested queries have this form

```
SELECT <column list>
FROM <a table>
WHERE <some columns> IN (
  SELECT <another column list>
  FROM <another table>
  WHERE <other condition>
)
```

- Write a query to display the department number and department name of all departments having some employees who have dependents
- Using nested queries and the above result, write an SQL statement that displays employee names, and department names for all departments satisfying the above condition
- Analyze the execution plan of the nested query
- Answer the same question but using **JOIN**. Compare the execution plans between the nested query and join statements

Exercise 4: Sargable Queries

Create an index on the **Bdate** field of the **Employee** table.

Write a query using the **YEAR()** function to retrieve all employees within a specific age range. Verify whether the index is used in the execution plan.

Then, rewrite the query so that the index on **Bdate** can be utilized effectively.

More resources for optimizing your queries

- SELECT optimisation
- WHERE optimisation
- Range optimisation
- Nested join optimisation
- Outer join optimisation
- Optimising subqueries
- Subquery optimisation
- IS NULL optimisation
- ORDER BY optimisation
- GROUP BY optimisation
- DISTINCT optimisation
- LIMIT optimisation
- Function optimisation
- Avoiding table scans

▶ Student Guide (If you need step by step guiding): click to expand